# torch.gradient#

https://pytorch.org/docs/stable/generated/torch.gradient.html#torch.gradient

**Description:**

Estimates the gradient of a function g:Rn→Rg : \mathbb{R}^n \rightarrow \mathbb{R}g:Rn→R in one or more dimensions using the second-order accurate central differences method and either first or second order estimates at the boundaries. The gradient of ggg is estimated using samples. By default, when spacing is not specified, the samples are entirely described by input, and the mapping of input coordinates to an output is the same as the tensor's mapping of indices to values. For example, for a three-dimensional input the function described is g:R3→Rg : \mathbb{R}^3 \rightarrow \mathbb{R}g:R3→R, and g(1,2,3) ==input[1,2,3]g(1, 2, 3)\ == input[1, 2, 3]g(1,2,3) ==input[1,2,3]. When spacing is specified, it modifies the relationship between input and input coordinates. This is detailed in the "Keyword Arguments" section below. The gradient is estimated by estimating each partial derivative of ggg independently. This estimation is accurate if ggg is in C3C^3C3 (it has at least 3 continuous derivatives), and the estimation can be improved by providing closer samples. Mathematically, the value at each interior point of a partial derivative is estimated using Taylor's theorem with remai

## Function Signature

```
torch.gradient(input, *, spacing=1, dim=None, edge_order=1)
→ List of Tensors#

Parameters

Keyword Arguments
```

## Parameters

### Keyword

spacing (scalar, list of scalar, list of Tensor, optional) – spacing can be used to modify how the input tensor's indices relate to sample coordinates. If spacing is a scalar then the indices are multiplied by the scalar to produce the coordinates. For example, if spacing=2 the indices (1, 2, 3) become coordinates (2, 4, 6). If spacing is a list of scalars then the corresponding indices are multiplied. For example, if spacing=(2, -1, 3) the indices (1, 2, 3) become coordinates (2, -2, 9). Finally, if spacing is a list of one-dimensional tensors then each tensor specifies the coordinates for the corresponding dimension. For example, if the indices are (1, 2, 3) and the tensors are (t0, t1, t2), then the coordinates are (t0[1], t1[2], t2[3])
dim (int, list of int, optional) – the dimension o

## Code Examples

```
>>> # Estimates the gradient of f(x)=x^2 at points [-2, -1, 2,
4]
>>> coordinates = (torch.tensor([-2., -1., 1., 4.]),)
>>> values = torch.tensor([4., 1., 1., 16.], )
>>> torch.gradient(values, spacing = coordinates)
(tensor([-3., -2., 2., 5.]),)

>>> # Estimates the gradient of the R^2 -> R function whose
samples are
>>> # described by the tensor t. Implicit coordinates are [0, 1]
for the outermost
>>> # dimension and [0, 1, 2, 3] for the innermost dimension,
and function estimates
>>> # partial derivative for both dimensions.
>>> t = torch.tensor([[1, 2, 4, 8], [10, 20, 40, 80]])
>>> torch.gradient(t)
(tensor([[ 9., 18., 36., 72.],
         [ 9., 18., 36., 72.]]),
 tensor([[ 1.0000, 1.5000, 3.0000, 4.0000],
         [10.0000, 15.0000, 30.0000, 40.0000]]))

>>> # A scalar value for spacing modifies the relationship
between tensor indices
>>> # and input coordinates by multiplying the indices to find
the
>>> # coordinates. For example, below the indices of the
innermost
>>> # 0, 1, 2, 3 translate to coordinates of [0, 2, 4, 6], and
the indices of
>>> # the outermost dimension 0, 1 translate to coordinates of
[0, 2].
>>> torch.gradient(t, spacing = 2.0) # dim = None (implicitly
[0, 1])
(tensor([[ 4.5000, 9.0000, 18.0000, 36.0000],
          [ 4.5000, 9.0000, 18.0000, 36.0000]]),
 tensor([[ 0.5000, 0.7500, 1.5000, 2.0000],
          [ 5.0000, 7.5000, 15.0000, 20.0000]]))
```

```
>>> # doubling the spacing between samples halves the estimated
partial gradients.

>>>
>>> # Estimates only the partial derivative for dimension 1
>>> torch.gradient(t, dim = 1) # spacing = None (implicitly 1.)
(tensor([[ 1.0000, 1.5000, 3.0000, 4.0000],
         [10.0000, 15.0000, 30.0000, 40.0000]]),)

>>> # When spacing is a list of scalars, the relationship
between the tensor
>>> # indices and input coordinates changes based on dimension.
>>> # For example, below, the indices of the innermost dimension
0, 1, 2, 3 translate
>>> # to coordinates of [0, 3, 6, 9], and the indices of the
outermost dimension
>>> # 0, 1 translate to coordinates of [0, 2].
>>> torch.gradient(t, spacing = [3., 2.])
(tensor([[ 4.5000, 9.0000, 18.0000, 36.0000],
         [ 4.5000, 9.0000, 18.0000, 36.0000]]),
 tensor([[ 0.3333, 0.5000, 1.0000, 1.3333],
         [ 3.3333, 5.0000, 10.0000, 13.3333]]))

>>> # The following example is a replication of the previous one
with explicit
>>> # coordinates.
>>> coords = (torch.tensor([0, 2]), torch.tensor([0, 3, 6, 9]))
>>> torch.gradient(t, spacing = coords)
(tensor([[ 4.5000, 9.0000, 18.0000, 36.0000],
         [ 4.5000, 9.0000, 18.0000, 36.0000]]),
 tensor([[ 0.3333, 0.5000, 1.0000, 1.3333],
         [ 3.3333, 5.0000, 10.0000, 13.3333]]))
```

## Notes & Warnings

NOTE

Note We estimate the gradient of functions in complex domain g:Cn→Cg : \mathbb{C}^n \rightarrow \mathbb{C}g:Cn→C in the same way.

## Detailed Documentation

### Documentation

Estimates the gradient of a function g:Rn→Rg : \mathbb{R}^n \rightarrow \mathbb{R}g:Rn→R in one or more dimensions using the second-order accurate central differences method and either first or second order estimates at the boundaries.

The gradient of ggg is estimated using samples. By default, when spacing is not specified, the samples are entirely described by input, and the mapping of input coordinates to an output is the same as the tensor's mapping of indices to values. For example, for a three-dimensional input the function described is g:R3→Rg : \mathbb{R}^3 \rightarrow \mathbb{R}g:R3→R, and g(1,2,3) ==input[1,2,3]g(1, 2, 3)\ == input[1, 2, 3]g(1,2,3) ==input[1,2,3].

When spacing is specified, it modifies the relationship between input and input coordinates. This is detailed in the "Keyword Arguments" section below.

The gradient is estimated by estimating each partial derivative of ggg independently. This estimation is accurate if ggg is in C3C^3C3 (it has at least 3 continuous derivatives), and the estimation can be improved by providing closer samples. Mathematically, the value at each interior point of a partial derivative is estimated using

Taylor’s theorem with remainder. Letting xxx be an interior point with x−hlx-h_lx−hl and x+hrx+h_rx+hr be points neighboring it to the left and right respectively, f(x+hr)f(x+h_r)f(x+hr) and f(x−hl)f(x-h_l)f(x−hl) can be estimated using:

Using the fact that f∈C3f \in C^3f∈C3 and solving the linear system, we derive:

We estimate the gradient of functions in complex domain g:Cn→Cg : \mathbb{C}^n \rightarrow \mathbb{C}g:Cn→C in the same way.

The value of each partial derivative at the boundary points is computed differently. See edge_order below.

input (Tensor) – the tensor that represents the values of the function

spacing (scalar, list of scalar, list of Tensor, optional) – spacing can be used to modify how the input tensor’s indices relate to sample coordinates. If spacing is a scalar then the indices are multiplied by the scalar to produce the coordinates. For example, if spacing=2 the indices (1, 2, 3) become coordinates (2, 4, 6). If spacing is a list of scalars then the corresponding indices are multiplied. For example, if spacing=(2, -1, 3) the indices (1, 2, 3) become coordinates (2, -2, 9). Finally, if spacing is a list of one-dimensional tensors then each tensor specifies the coordinates for the corresponding dimension. For example, if the indices are (1, 2, 3) and the tensors are (t0, t1, t2), then the coordinates are (t0[1], t1[2], t2[3])

dim (int, list of int, optional) – the dimension or dimensions to approximate the gradient over. By default the partial gradient in every dimension is computed. Note that when dim is specified the elements of the spacing argument must correspond with the specified dims.”

edge_order (int, optional) – 1 or 2, for first-order or second-order estimation of the boundary (“edge”) values, respectively. Note that when edge_order is specified, each dimension size of input should be at least edge_order+1